

REST API: Energy Bar Challenge

Problem Description

Given a country name, find the chocolate from that country that provides the most energy per gram using a chocolate database. You must access the database via HTTP GET requests to the following endpoint:

```
https://jsonmock.hackerrank.com/api/chocolates?countryOfOrigin={country}
```

The results are paginated. Append `&page={num}` to the URL to access specific pages.

Energy Calculation

For a given country, return the brand and type of chocolate in the format "**brand:type**" that provides the most energy per gram. Use the following formula:

$$\text{Energy} = \text{floor}(\text{kcal} \times 0.01 \times \text{average_weight})$$

- Round the energy calculation **down** to the nearest integer (floor).
- For chocolates with multiple weights, use the **average weight** rounded down.
- **Tie-breaking:** If multiple chocolates provide the same energy, return the one with the **lowest average price** (rounded down).

Sample Case 0

Input: France

Output: Patchi:Caramel Chocolate

Explanation: For the country France, Patchi Caramel Chocolate provides 384 kcal of energy at an average price of 382.

Solution (Python 3)

```
import requests
import math

def energyBar(country):
    base_url = f"https://jsonmock.hackerrank.com/api/chocolates?countryOfOrigin={country}"

    chocolates = []
    current_page = 1
    total_pages = 1

    # Fetch all paginated data
    while current_page <= total_pages:
        response = requests.get(f"{base_url}&page={current_page}")
        data = response.json()

        if current_page == 1:
            total_pages = data['total_pages']

        chocolates.extend(data['data'])
        current_page += 1

    best_chocolate = None
    max_energy = -1
    min_avg_price = float('inf')

    for chocolate in chocolates:
        # Calculate Average Weight
```

```

weights = chocolate.get('weights', [])
avg_weight = sum(weights) / len(weights) if weights else 0

# Calculate Energy: floor(kcal * 0.01 * avg_weight)
kcal = chocolate.get('nutritionalInformation', {}).get('kcal', 0)
energy = math.floor(kcal * 0.01 * avg_weight)

# Calculate Average Price
prices = chocolate.get('prices', [])
avg_price = math.floor(sum(prices) / len(prices)) if prices else 0

# Selection logic with tie-breaking
if energy > max_energy:
    max_energy = energy
    min_avg_price = avg_price
    best_chocolate = chocolate
elif energy == max_energy:
    if avg_price < min_avg_price:
        min_avg_price = avg_price
        best_chocolate = chocolate

if best_chocolate:
    return f"{best_chocolate['brand']}:{best_chocolate['type']}"
return ""

```